

Code:

```
#include <stdio.h> #include <stdlib.h>

#define MAXN 600005

int bit[MAXN];

void update(int i, int n, int x) { for (; i <= n; i += i & -i) { bit[i] += x; } }

int query(int i) { int sum = 0; for (; i > 0; i -= i & -i) { sum += bit[i]; } return sum; }

int compare(const void *a, const void *b) { return ((int *)a - *(int *)b); }

int bsearch_coord(int *vals, int sz, int x) { int l = 0, r = sz - 1, ans = -1; while (l <= r) { int mid = (l + r) / 2; if (vals[mid] >= x) { ans = mid; r = mid - 1; } else { l = mid + 1; } } return ans + 1; }

int bsearch_upper(int *vals, int sz, int x) { int l = 0, r = sz - 1, ans = -1; while (l <= r) { int mid = (l + r) / 2; if (vals[mid] <= x) { ans = mid; l = mid + 1; } else { r = mid - 1; } } return ans + 1; }

struct Query { char type; int a, b; };

int p[MAXN]; struct Query qarr[MAXN]; int coords[MAXN];

int main() { int n, q; if (scanf("%d %d", &n, &q) != 2) return 0;

    int c_cnt = 0;
    for (int i = 1; i <= n; i++) {
        scanf("%d", &p[i]);
        coords[c_cnt++] = p[i];
    }

    for (int i = 0; i < q; i++) {
        scanf(" %c %d %d", &qarr[i].type, &qarr[i].a, &qarr[i].b);
        if (qarr[i].type == '!') {
            coords[c_cnt++] = qarr[i].b;
        }
    }

    qsort(coords, c_cnt, sizeof(int), compare);
    int unique_c = 0;
    for (int i = 0; i < c_cnt; i++) {
        if (i == 0 || coords[i] != coords[i - 1]) {
            coords[unique_c++] = coords[i];
        }
    }
}
```

```

    }
}

for (int i = 1; i <= n; i++) {
    int idx = bsearch_coord(coords, unique_c, p[i]);
    update(idx, unique_c, 1);
}

for (int i = 0; i < q; i++) {
    if (qarr[i].type == '!') {
        int k = qarr[i].a;
        int new_val = qarr[i].b;
        int old_idx = bsearch_coord(coords, unique_c, p[k]);
        update(old_idx, unique_c, -1);
        p[k] = new_val;
        int new_idx = bsearch_coord(coords, unique_c, new_val);
        update(new_idx, unique_c, 1);
    } else {
        int a = qarr[i].a;
        int b = qarr[i].b;
        int left_idx = bsearch_coord(coords, unique_c, a);
        int right_idx = bsearch_upper(coords, unique_c, b);
        if (left_idx > right_idx || left_idx == 0 || right_idx == 0)
        {
            printf("0\n");
        } else {
            printf("%d\n", query(right_idx) - query(left_idx - 1));
        }
    }
}
return 0;
}

```

Input:

```

5 3
3 7 2 2 5
? 2 3
! 3 6
? 2 3

```

Output:

3

2